

① What is React?

- React is a JavaScript library used to build user interfaces, especially single page applications where you need a fast, dynamic and smooth user experience.



In Simple Words

React helps us build reusable UI components and manage the state of our application efficiently.

② Why React?

- Component Based Architecture
- Reusability of Components
- Virtual DOM (High Performance)
- Easy to Learn
- Huge Community Support
- Used by Top Companies

Popular Companies Using React



Facebook



Instagram



Netflix



Airbnb



WhatsApp



Tesla



Discord



Shopify

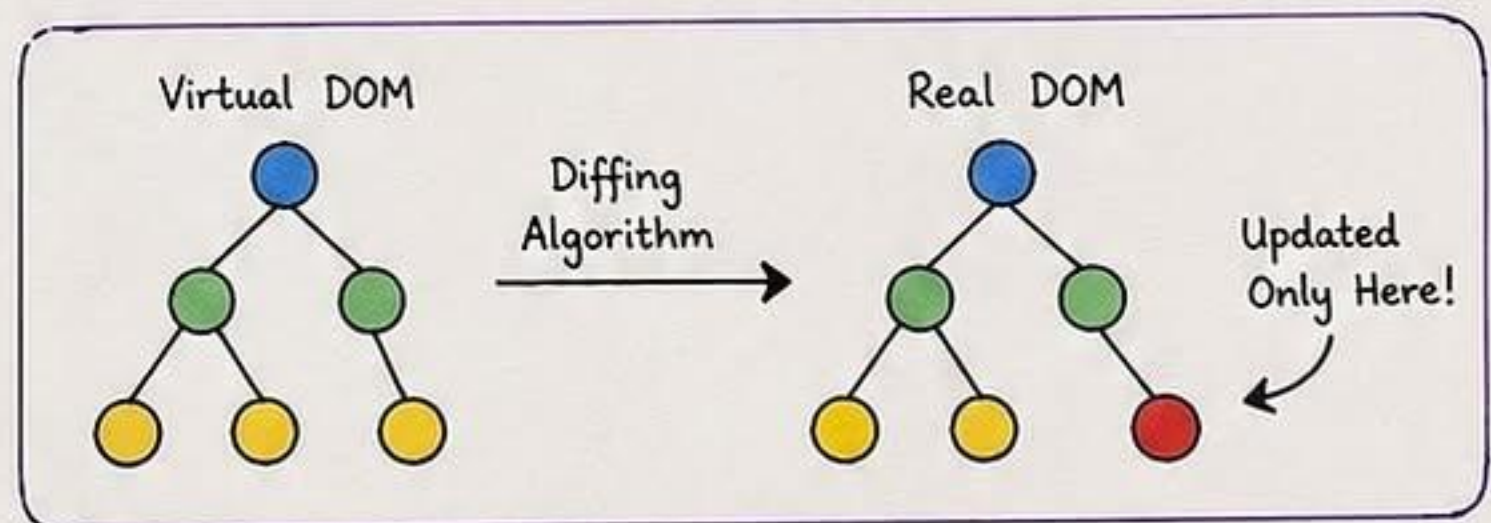
③ Features of React

- ✓ Declarative - You describe what UI you want, React handles how to update the DOM.
- ✓ Component Based - Build independent and reusable UI components.
- ✓ Virtual DOM - React creates a lightweight copy of the actual DOM and updates only what changed.

- ✓ Unidirectional Data Flow - Data flows in one direction which makes the app predictable.
- ✓ JSX - Write HTML inside JavaScript in a more readable way.
- ✓ Strong Ecosystem - Large collection of tools and libraries.

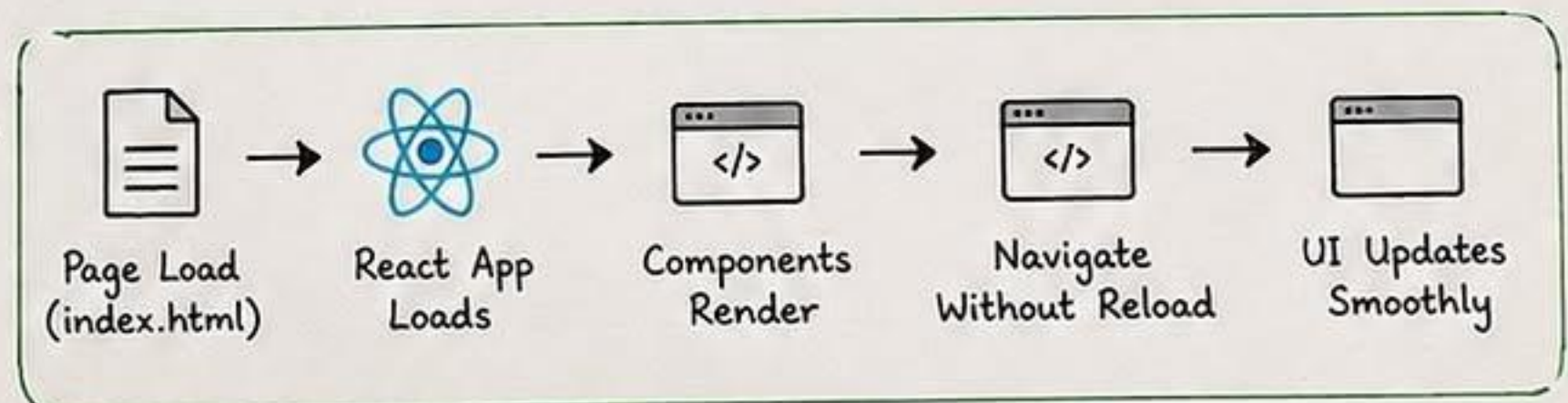
④ Virtual DOM

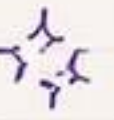
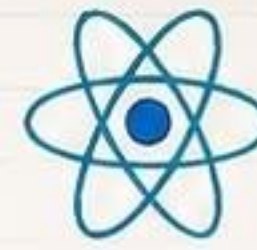
React uses a Virtual DOM which is a lightweight copy of the real DOM. When any data changes, React first updates the Virtual DOM, then efficiently updates only the changed parts in the Real DOM.



⑤ SPA (Single Page Application)

React is mainly used to create SPAs. In SPA, page does not reload when you navigate. Only the necessary components update, and it feels like a smooth app.



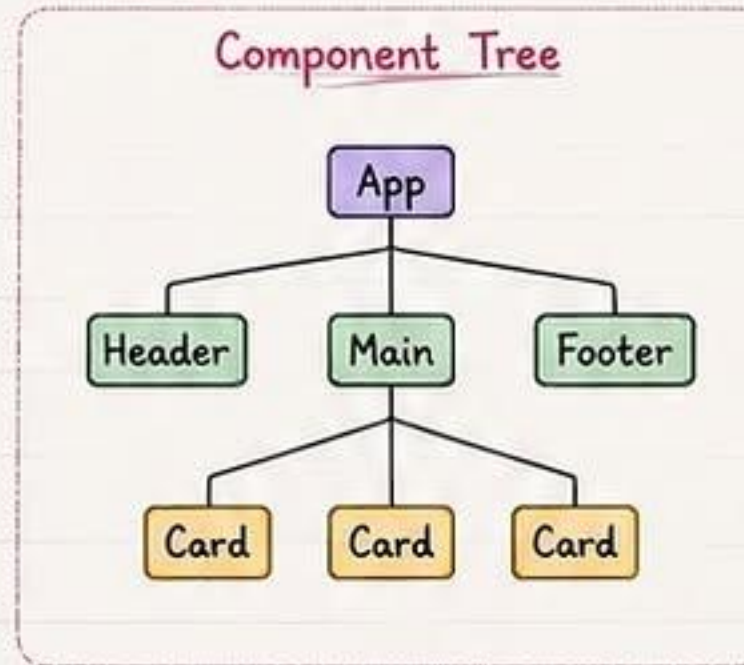


② React Fundamentals

These are the building blocks of any React application. ✨

① Components

- Components are the independent and reusable pieces of UI.
- A React app is made up of multiple components.
- Types :
 - ▶ Functional Components
 - ▶ Class Components (Old way)



② Functional vs Class Components

Functional Component

```
function Greet() {
  return <h1>Hello React!</h1>;
}
```

Class Component (Old way)

```
class Greet extends React.Component {
  render() {
    return <h1>Hello React!</h1>;
  }
}
```

③ JSX (JavaScript XML)

- JSX is a syntax extension for JavaScript.
- It looks like HTML but works inside JavaScript.
- JSX makes code more readable and easy to write.

```
const element = <h1 className="title">
  Hello, React!
</h1>;
```

↑
className in JSX instead of class

④ Props

- Props (short for properties) are read-only.
- Used to pass data from parent to child component.
- Props help in reusability of components.

Parent Component

```
function App() {
  return <User name="Zaid" age={25} />;
}
```

Child Component

```
function User(props) {
  return <p>{props.name}
    is {props.age} years old.
  </p>;
}
```

⑤ State

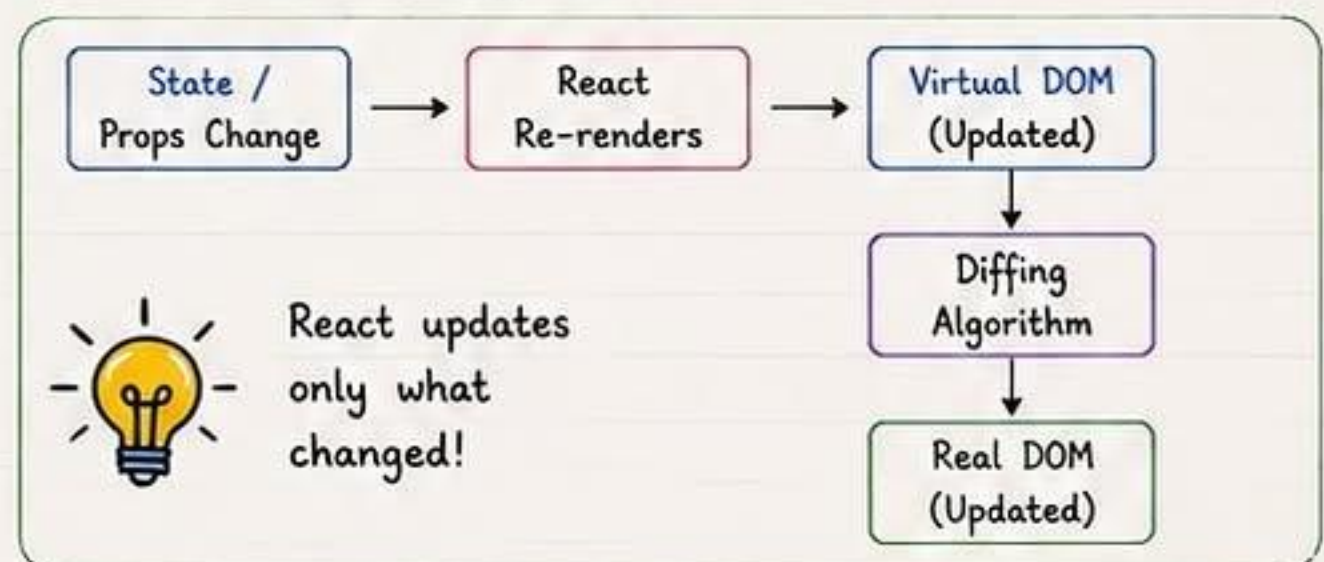
- State is a built-in object to store data that can change over time.
- When state changes, the component re-renders.
- Only used inside functional components using `useState` hook.

```
import { useState } from 'react';
function Counter() {
  const [count, setCount] = useState(0);
  return (
    <button onClick={() => setCount(count + 1)}>
      Count: {count}
    </button>
  );
}
```

- Initialize state
- Update state
- Re-renders on state change!

⑥ Rendering

- React renders components to the DOM.
- Whenever state or props change, React re-renders the component.



⑦ Event Handling

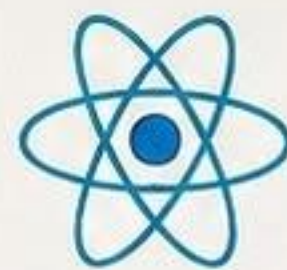
- Events are written in camelCase.
- Pass a function reference, not a string.
- Example: `onClick`, `onChange`, `onSubmit`

```
function ButtonClick() {
  const handleClick = () => {
    alert('Button Clicked!');
  };
  return <button onClick={handleClick}>
    Click Me
  </button>;
}
```

Key Takeaway

React is all about components, props and state.





"Hooks let you use state and other React features without writing a class."

③ React Hooks

Hooks are special functions that let you "hook into" React features like state and lifecycle from functional components. ✨

Benefit of Hooks

- ✓ Use state without classes
- ✓ Use lifecycle methods
- ✓ Reuse logic with custom Hooks
- ✓ Cleaner and easier code

① useState()

- useState lets you add state to a functional component.
- It returns a state value and a function to update it.

Example :

```
import { useState } from 'react';

function Counter() {
  const [count, setCount] = useState(0);

  return (
    <div>
      <h2>Count: {count}</h2>
      <button onClick={() => setCount(count + 1)}>
        Increment
      </button>
    </div>
  );
}
```

state variable

function to update state

② useEffect()

- useEffect lets you perform side effects in functional components.
- Runs after every render by default.
- You can control when it runs.

Example :

```
import { useState, useEffect } from 'react';

function Example() {
  const [count, setCount] = useState(0);

  useEffect(() => {
    console.log('Component Rendered');
  }); // runs after every render

  return (
    <button onClick={() => setCount(count + 1)}>
      Clicked {count} times
    </button>
  );
}
```



Note :

- [] → runs only once (like componentDidMount)
- [count] → runs when 'count' changes
- No array → runs after every render

③ useRef()

- useRef returns a mutable ref object.
- Changes in ref do not cause re-render.
- Commonly used to access DOM elements.

Example :

```
import { useRef } from 'react';

function FocusInput() {
  const inputRef = useRef(null);

  const focusInput = () => {
    inputRef.current.focus();
  };

  return (
    <div>
      <input ref={inputRef} type="text"
        placeholder="Focus me" />
      <button onClick={focusInput}> Focus</button>
    </div>
  );
}
```

ref object

④ useMemo()

- useMemo memorizes a value.
- Expensive calculations are only recomputed when dependencies change.

Example :

```
import { useMemo } from 'react';

function Expensive({ num }) {
  const result = useMemo(() => {
    console.log('Calculating...');
    return num * 2;
  }, [num]); // only when num changes

  return <h2>Result: {result}</h2>;
}
```

Memoized value

dependencies array

⑤ useCallback()

- useCallback returns a memoized function.
- Useful when passing functions to child components to prevent unnecessary re-renders.

Example :

```
import { useState, useCallback } from 'react';

function Parent() {
  const [count, setCount] = useState(0);

  const handleClick = useCallback(() => {
    console.log('Clicked!');
  }, []); // memoized function

  return <Child onClick={handleClick} />;
}
```

memoized function

⑥ Custom Hooks

- You can create your own Hooks to reuse logic.
- Custom Hooks start with "use".

Example :

```
// useWindowSize.js
import { useState, useEffect } from 'react';

export function useWindowSize() {
  const [size, setSize] = useState({
    width: window.innerWidth,
    height: window.innerHeight
  });

  useEffect(() => {
    const handleResize = () => {
      setSize({ width: window.innerWidth, height: window.innerHeight });
    };
    window.addEventListener('resize', handleResize);
    return () => window.removeEventListener('resize', handleResize);
  }, []);

  return size;
}

// Usage in a component
const { width, height } = useWindowSize();
```

Reusable logic

Rules of Hooks

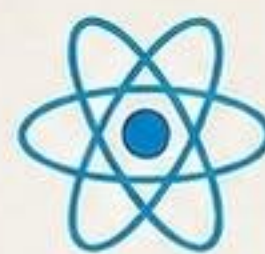
- ① Only call Hooks at the top level.
- ② Do not call Hooks inside loops, conditions or nested functions.
- ③ Only call Hooks from React functional components or custom Hooks.

Commonly Used Built-in Hooks

- useState
- useEffect
- useRef
- useMemo
- useCallback
- useContext
- useReducer
- useLayoutEffect

Remember

Hooks make your code clean, reusable and powerful! 🌟



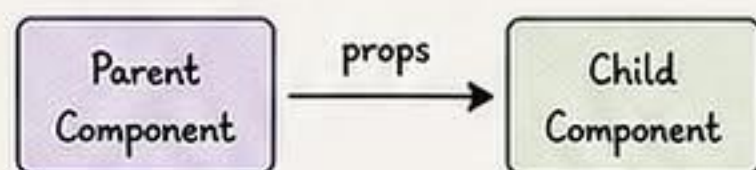
④ Component Communication

React is all about building tiny, reusable components.

Components need to communicate to share data and trigger actions. ✨

① Parent → Child Communication

- Parent passes data to child using props.
- Read-only for child.



Example:

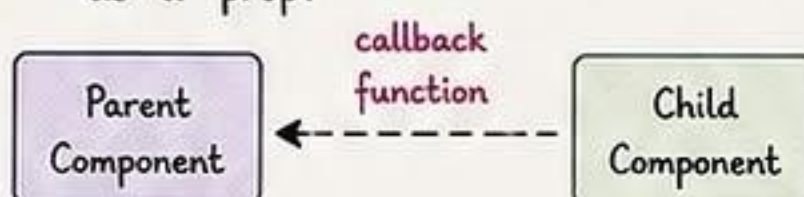
```

function Parent() {
  return <Child name="Zaid" age={25} />;
}

function Child(props) {
  return <h2>{props.name} - {props.age}</h2>;
}
  
```

② Child → Parent Communication

- Child sends data to parent using callback function.
- Parent passes the function to child as a prop.



Example:

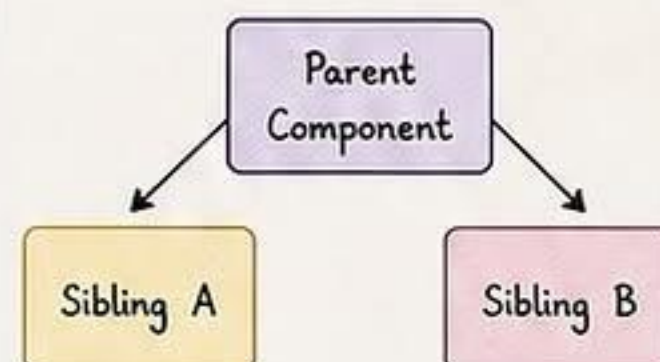
```

function Parent() {
  const handleData = (data) => {
    console.log(data);
  };
  return <Child sendData={handleData} />;
}

function Child({ sendData }) {
  return <button onClick={() => sendData("Hi Parent!")}>Send</button>;
}
  
```

③ Sibling Communication

- Siblings cannot communicate directly.
- Share data via their common parent.

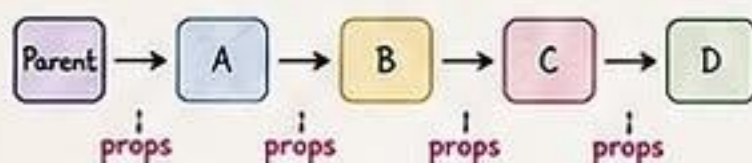


Example:

- ☑ Parent holds the state.
- ☑ Updates are passed as props to siblings.

④ Props Drilling

- Passing props through many layers of components.
- Can make code hard to maintain.
- Avoid by using Context API or state management libraries.



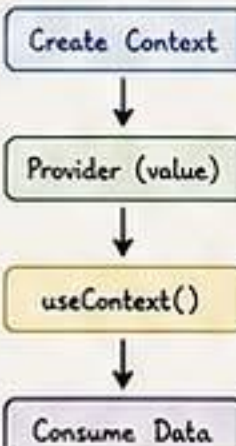
⑤ Context API

- Share data across the component tree without passing props manually.
- Ideal for global data like theme, user, language, etc.

Steps:

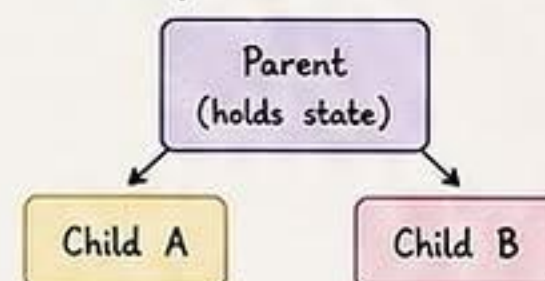
- ① Create Context
- ② Wrap with Provider
- ③ Consume with useContext()

Simple Flow



⑥ Lifting State Up

- Move the state to the closest common parent.
- Helps in sharing state between multiple children.



☑ Both children re-render on state change.

⑦ Controlled vs Uncontrolled Components

Controlled Components

- ☑ Form data is handled by React state.
- ☑ Value comes from state.
- ☑ More control and validation.

Example:

```

const [name, setName] = useState("");
<input value={name} onChange={(e) => setName(e.target.value)} />
  
```

Uncontrolled Components

- ☑ Form data is handled by DOM.
- ☑ Use refs to get values.
- ☑ Less code, but less control.

Example:

```

const nameRef = useRef();
<input ref={nameRef} />
<button onClick={() => console.log(nameRef.current.value)}>Get Value</button>
  
```

Key Takeaway



- ☑ Props down, events up.
- ☑ Keep components small and focused.
- ☑ Use Context API or state management for global data.
- ☑ Lift state up for better communication.



When to Use What?



Props

For simple data passing
(Parent → Child)



Callback

For Child → Parent
communication



Context API

For global data sharing
(across many components)



Lift State Up

When multiple siblings need
the same state



5 Routing & Navigation

React Router is the standard library for routing in React applications. It helps us build Single Page Applications (SPAs) with multiple views. ✨

Why Routing?

- ✓ Navigate between pages without reload
- ✓ Better user experience
- ✓ Bookmarkable URLs
- ✓ Build complex, nested applications

1 React Router Setup

- First, install the library:

```
npm install react-router-dom
```

- Wrap your app with BrowserRouter

```
import { BrowserRouter } from 'react-router-dom';
import App from './App';

function Root() {
  return (
    <BrowserRouter>
      <App />
    </BrowserRouter>
  );
}
export default Root;
```

2 Routes & Route

- <Routes> is a container for all routes.
- <Route> defines a path and what component to render.

```
import { Routes, Route } from 'react-router-dom';
import Home from './pages/Home';
import About from './pages/About';
import Contact from './pages/Contact';

function App() {
  return (
    <Routes>
      <Route path="/" element={<Home />} />
      <Route path="/about" element={<About />} />
      <Route path="/contact" element={<Contact />} />
    </Routes>
  );
}
export default App;
```

3 Link vs NavLink

Link - Used for simple navigation.

```
<Link to="/about">About</Link>
```

NavLink - Adds active styles when the link is active.

Perfect for navigation menus.

```
<NavLink to="/" className={({ isActive }) =>
  isActive ? 'active' : ''
}>
  Home
</NavLink>
```



Tip: NavLink automatically applies active class when the route matches.

4 useNavigate Hook

- Programmatic navigation.
- Used to navigate in code (on button click, form submit, etc.).

```
import { useNavigate } from 'react-router-dom';

function Dashboard() {
  const navigate = useNavigate();

  const handleLogout = () => {
    navigate('/login'); // navigate to login
  };

  return (
    <button onClick={handleLogout}>Logout</button>
  );
}
```

5 Route Parameters

- Use dynamic segments in the URL.
- Access params using useParams().

```
// Route
<Route path="/user/:id" element={<User />} />

// Component
import { useParams } from 'react-router-dom';

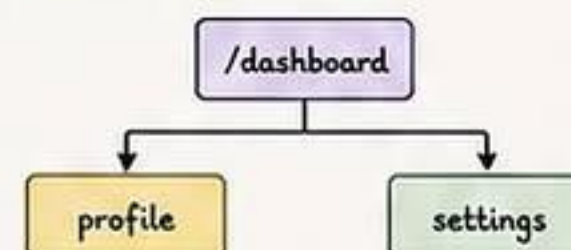
function User() {
  const { id } = useParams();

  return <h2>User ID: {id}</h2>;
}
```

6 Nested Routes

- Routes inside other routes.
- Useful for layouts and dashboards.

```
<Route path="/dashboard" element={<Dashboard />}>
  <Route path="profile" element={<Profile />} />
  <Route path="settings" element={<Settings />} />
</Route>
```



7 Protected Routes

- Restrict access to certain routes.
- Check authentication before rendering.

```
function RequireAuth({ children }) {
  const isAuthenticated = localStorage.getItem('token');
  return isAuthenticated ? children : <Navigate to="/login"/>;
}

<Route path="/dashboard" element={
  <RequireAuth>
    <Dashboard />
  </RequireAuth>
} />
```



8 404 - Not Found Page

- Add a wildcard route (*) at the end.
- It matches any unknown route.

```
<Routes>
  <Route path="/" element={<Home />} />
  <Route path="/about" element={<About />} />
  <Route path="*" element={<NotFound />} />
</Routes>
```



Always keep the * route last, otherwise it will match everything!

9 useLocation Hook

- Access information about the current URL.
- Useful for analytics, breadcrumbs, etc.

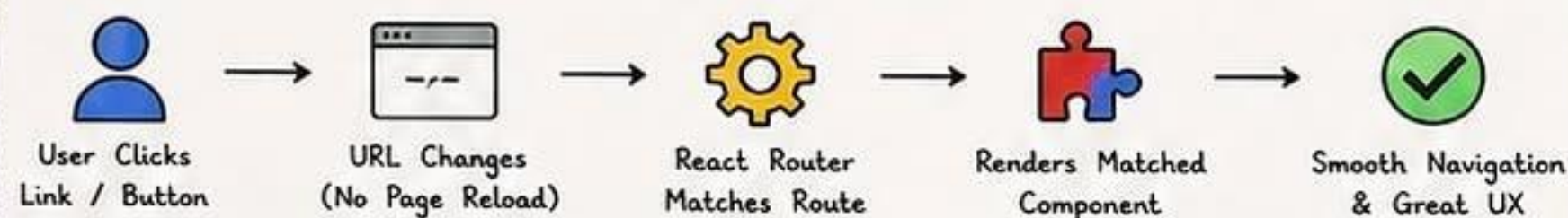
```
import { useLocation } from 'react-router-dom';

function PageTracker() {
  const location = useLocation();

  useEffect(() => {
    console.log('Current Path:', location.pathname);
  }, [location]);

  return null;
}
```

React Router Flow



Remember



- ✓ Use <Link> for navigation
- ✓ Use <NavLink> for menus
- ✓ useNavigate() for programmatic nav
- ✓ Always add 404 route
- ✓ Protect important routes



6 State Management

State is the heart of any React application.

It represents data that can change over time. ✨

Types of State

- Local State - Managed within a component
- Global State - Shared across the app
- Server State - Fetched from server & cached

1 Local State (useState)

- Used in individual components.
- Simple and easy to manage.

Example :

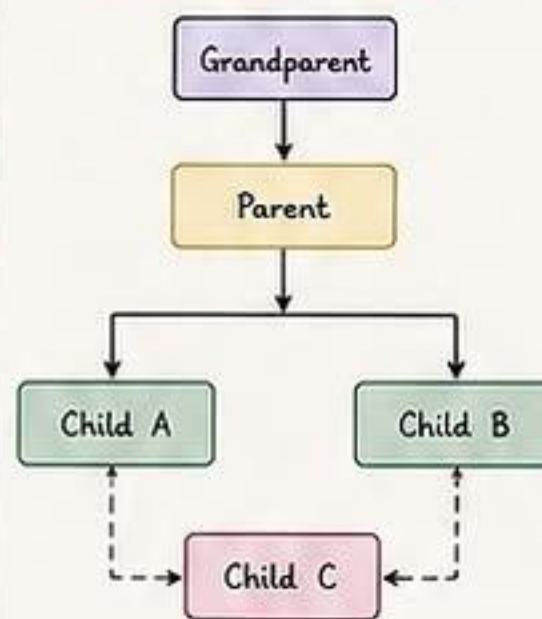
```
import { useState } from 'react';
function Counter() {
  const [count, setCount] = useState(0);
  return (
    <div>
      <h2>Count: {count}</h2>
      <button onClick={() => setCount(count + 1)}>
        Increment
      </button>
    </div>
  );
}
```

State is private to this component

2 Global State (Need)

- When many components need the same data, we lift the state up or use global state.
- Avoid props drilling.

Problem (Props Drilling)



Passing props through many levels is hard to maintain.

3 Context API

- Built-in React feature for global state.
- Great for theme, user, language etc.
- Avoids props drilling.

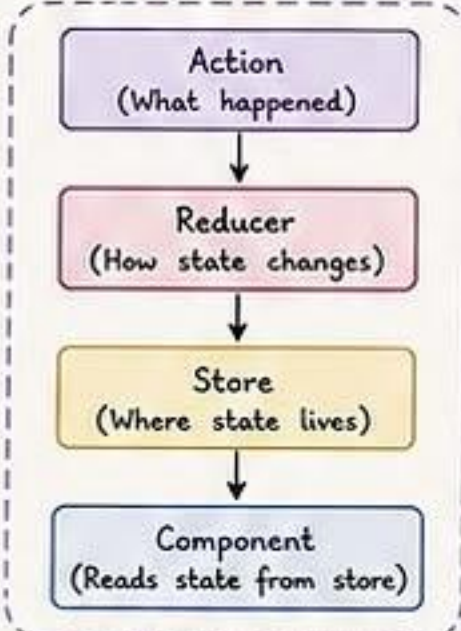
Example :

```
const ThemeContext = createContext();
function App() {
  return (
    <ThemeContext.Provider value="dark">
      <Header />
    </ThemeContext.Provider>
  );
}
// In any child component
const { theme } = useContext(ThemeContext);
```

4 Redux (Toolkit)

- Predictable state container.
- Best for large & complex apps.
- Uses store, actions, reducers.

Example (Basic Flow):



5 Zustand

- Lightweight global state library.
- Simple API. Less boilerplate than Redux.

Example :

```
import { create } from 'zustand';
const useStore = create((set) => ({
  count: 0,
  increment: () => set((state) => ({
    count: state.count + 1,
  })),
}));
// In component
const { count, increment } = useStore();
```

6 Recoil

- State management library by Facebook.
- Atom based.
- Good for complex state logic.

Example :

```
import { atom, useRecoilState } from 'recoil';
const countState = atom({
  key: 'countState',
  default: 0,
});
function Counter() {
  const [count, setCount] = useRecoilState(countState);
}
```

7 When to Use What?

Use Case	Best Choice
Simple component state	useState
Share state in few components	Context API
Large app with complex state	Redux Toolkit
Lightweight & easy global state	Zustand
Complex state with derived data	Recoil

8 Server State (React Query)

- Used for data from server.
- Handles caching, syncing, refetching.
- Makes API calls easy.

Example :

```
import { useQuery } from '@tanstack/react-query';
function Users() {
  const { data, isLoading, error } = useQuery({
    queryKey: ['users'],
    queryFn: () => fetch('/api/users').then(res => res.json())
  });
}
```

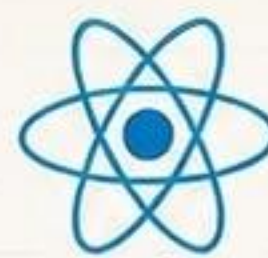
Benefits

- ✓ Caching
- ✓ Background refetching
- ✓ Loading & error handling
- ✓ Sync data automatically
- ✓ Better performance

Key Takeaways

- ✓ Use useState for local state.
- ✓ Use Context API for simple global state.
- ✓ Use Redux Toolkit for large, scalable apps.
- ✓ Use Zustand for lightweight global state.
- ✓ Use React Query for server state management.
- ✓ Choose the right tool based on your app's complexity.





7 API Integration

APIs let our React apps talk to the outside world and bring data to life. ✨

Popular Tools

- Fetch API - Built-in
- Axios - Easy & feature rich
- React Query - Data fetching made simple

1 Fetch API (Built-in)

- Native browser API.
- Promise based.
- Requires manual error handling.

Example :

```
fetch('https://jsonplaceholder.typicode.com/posts')
  .then(res => res.json())
  .then(data => console.log(data))
  .catch(err => console.error(err));
```

💡 Tip: Always handle errors in fetch.

2 Axios

- Promise based HTTP client.
- Automatically transforms JSON data.
- Cleaner syntax.

Example :

```
import axios from 'axios';
axios.get('https://jsonplaceholder.typicode.com/posts')
  .then(res => console.log(res.data))
  .catch(err => console.error(err));
```

💡 Tip: Axios throws error for non-2xx status codes.

3 Async / Await

- Cleaner way to write async code.
- Easy to read & maintain.

Example :

```
async function getPosts() {
  try {
    const res = await fetch(
      'https://jsonplaceholder.typicode.com/posts'
    );
    const data = await res.json();
    console.log(data);
  } catch (error) {
    console.error(error);
  }
}
getPosts();
```

4 Loading State

- Show loader while data is being fetched.
- Improves user experience.

Example :

```
const [loading, setLoading] = useState(true);
const [data, setData] = useState(null);
useEffect(() => {
  async function fetchData() {
    setLoading(true);
    const res = await fetch(API_URL);
    const result = await res.json();
    setData(result);
    setLoading(false);
  }
  fetchData();
}, []);
return loading ? <p>Loading...</p> : <DataView data={data}/>;
```

5 Error Handling

- Always handle API errors gracefully.
- Show meaningful messages to users.

Example :

```
const [error, setError] = useState(null);
try {
  const res = await fetch(API_URL);
  if (!res.ok) throw new Error('Something went wrong');
  const data = await res.json();
  setData(data);
} catch (err) {
  setError(err.message);
}
```

★ Best Practice: Show user friendly error, not raw error.

6 CRUD Operations (Example with Axios)

- Create - POST
- Read - GET
- Update - PUT / PATCH
- Delete - DELETE

Examples :

```
// CREATE
axios.post(API_URL, newData);
// READ
axios.get(API_URL);
// UPDATE
axios.put(`${API_URL}/${id}`, updatedData);
// DELETE
axios.delete(`${API_URL}/${id}`);
```

7 Custom API Hook

- Reuse API logic across components.
- Keep components clean.

Example (useFetch.js)

```
import { useState, useEffect } from 'react';
export default function useFetch(url) {
  const [data, setData] = useState(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);
  useEffect(() => {
    async function fetchData() {
      try {
        const res = await fetch(url);
        if (!res.ok) throw new Error('Failed to fetch');
        const json = await res.json();
        setData(json);
      } catch (err) {
        setError(err.message);
      } finally {
        setLoading(false);
      }
    }
    fetchData();
  }, [url]);
  return { data, loading, error };
}
```

Usage in Component

```
import useFetch from './useFetch';
function PostList() {
  const { data, loading, error } = useFetch(
    'https://jsonplaceholder.typicode.com/posts'
  );
  if (loading) return <p>Loading...</p>;
  if (error) return <p>Error: {error}</p>;
  return (
    <ul>
      {data.map(post => (
        <li key={post.id}>{post.title}</li>
      ))}
    </ul>
  );
}
```

Best Practices

- ✓ Keep API calls in services or custom hooks.
- ✓ Always handle loading & error states.
- ✓ Use environment variables for API URLs.
- ✓ Avoid making API calls inside render.
- ✓ Use React Query for complex data needs.

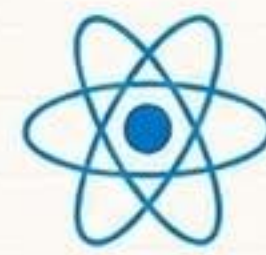
Quick Comparison

Feature	Fetch API	Axios	React Query
Built-in	Yes	No	No
Ease of Use	Medium	Easy	Very Easy
Error Handling	Manual	Automatic (2xx)	Automatic
Caching	No	No	Yes
Best For	Small apps	Most apps	Large apps



Remember: Data fetching is 20% code, 80% good UX.





⑧ Performance Optimization

Optimize your React apps to make them fast, smooth and user-friendly. ✨

Why Optimize?

- ✓ Better User Experience
- ✓ Faster Load Time
- ✓ Lower Resource Usage
- ✓ Higher SEO Score



① React.memo

- Prevents re-rendering of component if props have not changed.
- Works only for functional components.

Example :

```
const User = React.memo(({ name, age }) => {
  console.log("User Rendered");
  return (
    <div className="user">
      <h3>{name}</h3>
      <p>{age}</p>
    </div>
  );
});
```



Use for pure components.

② useMemo

- Memoizes a value.
- Recomputes only when dependencies change.

Example :

```
const total = useMemo(() => {
  console.log("Calculating total");
  return items.reduce((sum, item) => sum + item.price, 0);
}, [items]);
```



Use for expensive calculations.

③ useCallback

- Memoizes a function.
- Prevents unnecessary re-creation of functions.

Example :

```
const handleClick = useCallback(() => {
  console.log("Button Clicked");
}, [id]);
```



Useful when passing functions to child components.

④ Lazy Loading (React.lazy)

- Loads component only when needed.
- Reduces initial bundle size.

Example :

```
const Dashboard = React.lazy(() =>
  import("../Dashboard")
);
<Suspense fallback={<p>Loading..., </p>}>
  <Dashboard />
</Suspense>
```



Use Suspense as a fallback UI.

⑤ Code Splitting

- Splits code into smaller chunks.
- Improves load performance.

Example :

```
// Route based code splitting with React Router
const About = React.lazy(() => import("../pages/About"));
const Contact = React.lazy(() => import("../pages/Contact"));

<Suspense fallback={<p>Loading..., </p>}>
  <Routes>
    <Route path="/about" element={<About />} />
    <Route path="/contact" element={<Contact />} />
  </Routes>
</Suspense>
```

⑥ Suspense

- Displays fallback UI while waiting for component / data.
- Works with lazy loading & data fetching.

Example :

```
<Suspense fallback={<h3>Loading..., </h3>}>
  <LazyComponent />
</Suspense>
```



Improves perceived performance.

⑦ Debouncing

- Delays execution until user stops triggering the event.
- Useful for search inputs, resize, scroll, etc.

Example :

```
const debounce = (func, delay) => {
  let timer;
  return (...args) => {
    clearTimeout(timer);
    timer = setTimeout(() => func(...args), delay);
  };
};

const handleSearch = debounce((value) => {
  console.log("Searching...", value);
}, 500);
```

API call will be made only after user stops typing.

⑧ Virtualization (Large Lists)

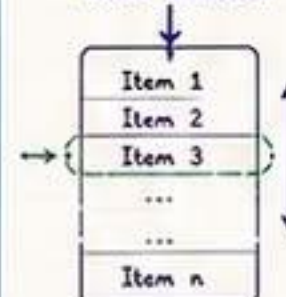
- Render only visible items in the list.
- Greatly improves performance for long lists.
- Popular Library: react-window / react-virtualized

Example (react-window):

```
import { FixedSizeList as List } from "react-window";

<List
  height={400}
  itemCount={items.length}
  itemSize={50}
  width={"100%"}
>
  {Row}
</List>
```

Only Visible Items Rendered



⑨ Avoid Unnecessary Re-renders

- Don't pass new objects / arrays / functions in props.
- Use stable references.

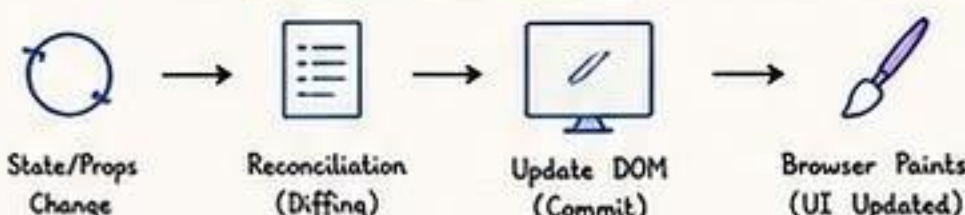
Bad ✗

```
<Child data={{ name: "Zaid" }} />
// new object every render
```

Good ✓

```
const user = useMemo(() => ({ name: "Zaid" }), []);
<Child data={user} />
// same reference
```

How React Renders?



☆ Optimize re-renders to make this cycle faster!

Tools for Performance

- ✓ React DevTools Profiler
- ✓ Lighthouse (Chrome)
- ✓ Web Vitals (Performance Metrics)
- ✓ why-did-you-render (Library)

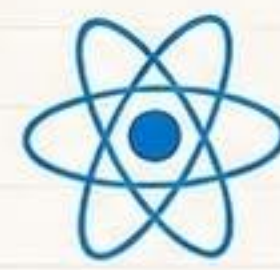
Remember ♥

- ✓ Measure first, then optimize.
- ✓ Don't over-optimize.
- ✓ Write clean & readable code.
- ✓ User experience > Everything.



Golden Rule : Optimize what matters, not everything.





9 React Project Structure

A well-structured project is easy to scale, maintain and collaborate. ✨

1 Recommended Folder Structure (Feature Based)

my-react-app/



2 Folder Descriptions

Folder / File	Purpose
public/	Contains index.html and static assets.
assets/	Images, fonts, icons used in the app.
components/	Small, reusable UI components.
pages/	Different pages / routes in the app.
features/	Group related components, logic, API etc.
hooks/	Reusable custom hooks.
services/	API calls and external service handlers.
store/	Global state management (Redux / Zustand).
utils/	Helper functions and utilities.
constants/	Constant values used across the app.
styles/	Global CSS, variables, themes.
App.jsx	Root component that wraps the application.
main.jsx	Renders the React app to the DOM.

3 Environment Variables (.env)

Store secrets and config outside the code.

Example (.env)

```

VITE_API_BASE_URL=https://api.example.com
VITE_APP_NAME=ReactApp
VITE_ENV=development
VITE_TIMEOUT=10000
  
```



4 Best Practices

- ✓ Keep components small and focused.
- ✓ Use meaningful names for files and folders.
- ✓ Follow feature-based structure for large apps.
- ✓ Keep UI and business logic separate.
- ✓ Reuse components and hooks.
- ✓ Follow consistent code formatting (Prettier / ESLint).
- ✓ Remove unused code and dependencies.

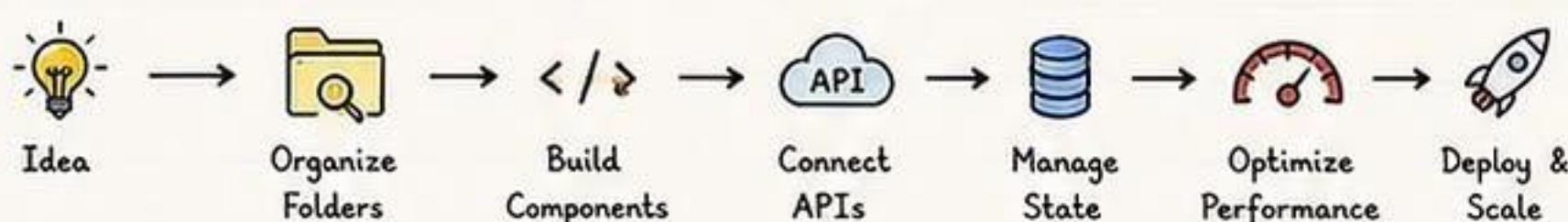
5 Naming Conventions

- **Components** : PascalCase (e.g., UserCard.jsx)
- **Hooks** : camelCase with "use" prefix (e.g., useAuth.js)
- **Files/Variables** : camelCase (e.g., userService.js)
- **Constants** : UPPER_SNAKE_CASE (e.g., API_URL)
- **Folders** : kebab-case (e.g., user-profile/)

6 Helpful Tips

- ☆ Use absolute imports with path alias (@/components/Button).
- ☆ Place components close to where they are used.
- ☆ Keep your project clean and modular.
- ☆ Document your code when needed.
- ☆ Commit small, meaningful changes.

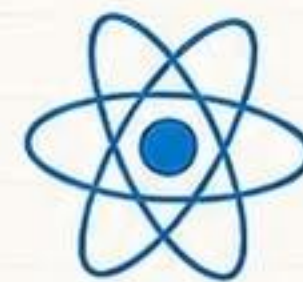
Project Structure Flow



Remember ♥

- ✓ Clean code = Happy future you.
- ✓ Structure today, scale tomorrow.
- ✓ Always write code that others can understand.





10 Deployment & Next Steps

You built it locally, now make it live for the world! 🌍 ✨

Popular Deployment Platforms ☁

- Vercel (Best for React) ▲ Vercel
- Netlify 🌐 netlify
- Firebase Hosting 🔥 Firebase
- GitHub Pages 🐙 GitHub Pages

1 Build for Production

- Optimize and create production build.
- Minify, bundle, tree-shake.

Commands :

```
npm run build
# or
yarn build
```

💡 Tip: Always test production build locally first.

2 Deploy (Example with Vercel)

- Connect your GitHub repo.
- Vercel auto-detects React & deploys.
- Every push = New deployment.

Steps :

- 1 Push code to GitHub
- 2 Import repo in Vercel
- 3 Click Deploy
- 4 Get live URL 🚀

💡 Tip: Use .env in Vercel dashboard (never commit secrets).

3 Post Deployment Checklist

- ☒ Test all core flows
- ☒ Check all routes
- ☒ Verify API integrations
- ☒ Setup custom domain (optional)
- ☒ Enable HTTPS
- ☒ Add favicon & meta tags
- ☒ Monitor performance & errors

💡 Tip: Monitor your app with Sentry / LogRocket / Vercel Analytics.

What's Next?

State Management



- Deep dive into Redux Toolkit
- Zustand for lighter apps

Testing



- Unit Testing (Jest)
- Component Testing (React Testing Library)

TypeScript



- Type-safe components
- Better DX & less bugs

Performance



- Code Splitting
- Memoization
- React Profiler

Advanced Topics



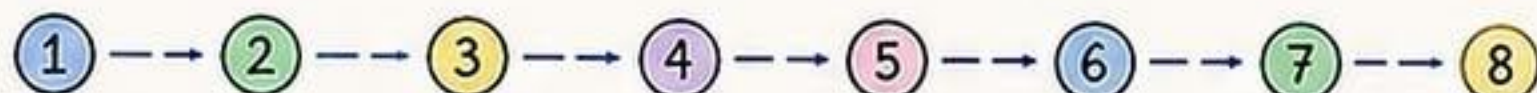
- Custom Hooks
- Context API
- Render Patterns
- Error Boundaries

Build Cool Stuff 🚀



- Real world projects
- Open Source
- Never stop building!

React Developer Roadmap (Summary)



1 Basics	2 Core Concepts	3 Routing	4 State Mgmt	5 Data & API	6 Performance	7 Testing	8 Deploy
JSX, Components, Props, State	Hooks, Events, Lists, Forms	React Router, Nested Routes	Context API, Redux / Zustand	Fetch, Axios, Async, CRUD	Optimize, Lazy, Memo, Profiler	Jest, RTL, Test Cases	Build, Deploy, Monitor

Keep Building. Keep Learning. Keep Shipping. ♥

The more you build, the better you get! 🚀



Remember: Consistency beats talent. Code a little every day.



Final Notes ♥

- ☒ Practice by building real projects.
- ☒ Read docs. A lot.
- ☒ Break things. Fix things.
- ☒ Ask "Why?" behind everything.
- ☒ Teach others - it makes you stronger.

You got this! 🧡

